Hamro Hospital — Full Technical Specification and Build Guide

This is a complete, end-to-end technical blueprint to build Hamro Hospital using Python, Django (LTS), Bootstrap 5, HTML5, CSS3, JavaScript, and PostgreSQL. You can paste this into AI coding tools to scaffold and implement the entire system, or hand it to a development team to implement directly. It includes architecture, database design, Django apps, models, views, URLs, forms, templates, auth, permissions, APIs, UI, workflows, validations, error handling, security, reports, notifications, backup, deployment, and rich seed data.

Note: You asked for a "clear documentation PDF." I can't attach files here, but you can copy this document into any editor and export it to PDF. Diagrams are provided in Mermaid format so you can render them in tools like Mermaid Live or VS Code plugins.

Time zone: Asia/Kathmandu
Locales: English (en), Nepali (ne) with i18n readiness

Target stack at project start (verify latest stable versions before implementation):

- **Python 3.11+**
- **Django 4.2 LTS**
- **PostgreSQL 15/16**
- **Redis 7+ (caching, Celery broker)**
- **Bootstrap 5.3+**
- **Node 18+ (optional for SCSS build), or use CDN**
- **Nginx + Gunicorn (production)**
- **Celery + Celery Beat (async tasks and scheduled jobs)**
- **WeasyPrint (PDF), qrcode and python-barcode (QR/Barcode)**
- **DRF (Django REST Framework)**
- **django-allauth (Google auth for patients)**
- **Jazzmin (Django Admin theme)**
- **django-axes or throttling middleware (anti-brute-force)**
- **django-environ (env handling)**
- **django-guardian (optional object-level perms)**
- **django-notifications-hq or custom notifications**
- **Django Channels (optional for real-time notifications)**

Branding: You will supply the hospital logo.

- **Primary color: the blue inside your logo**
- **Accent color (only for highlights/alerts/special buttons): the red in your logo**
- **Avoid red for normal buttons. Keep interface balanced, modern, clean, and professional.**

Dark Mode: Use Bootstrap 5 color modes and persist state via localStorage. Apply globally (public and dashboard).

1) System Overview and Architecture

Hamro Hospital has two major surfaces:

- **Public website: marketing pages, services, departments, doctors, patient self-registration and login.**
- **Secure dashboard: role-based application for Super Admin, Doctors, Nurses, Registration Counter, Pharmacy, Laboratory, Radiology, Blood Bank, Operation Theater, Admission & Discharge, Cash Counter, Accounts, Insurance, Medical Records, and Patients.**

Core principles:

- **RBAC using Django's Groups/Permissions with a custom User model.**
- **Audited, validated, and secure data flows.**
- **Every patient has a permanent barcode and QR code, used throughout the hospital.**
- **Doctors manage their schedule, quota, availability, and leave.**
- **Referrals flow automatically to target departments.**
- **Billing supports cash, eSewa, and QR payments with printable/downloadable PDFs.**
- **Scalable, maintainable modular Django apps.**

Mermaid architecture diagram (high-level):

mermaid

flowchart TB

subgraph PublicSite[Public Website]

Home

Services

Departments

DoctorsPublic

PatientReg

```
  Login
end

subgraph Backend[Django Backend + DRF]
  Accounts
  Patients
  Doctors
  Appointments
  Admissions
  Pharmacy
  Laboratory
  Radiology
  BloodBank
  OT
  Billing
  Insurance
  MedicalRecords
  Referrals
  Notifications
  Reports
  Settings
end

subgraph Infra[Infra]
  PostgreSQL[(PostgreSQL)]
  Redis[(Redis)]
  Media[(S3/Local Media)]
  Celery[Celery & Beat]
```

WeasyPrint[PDF]

end

PublicSite <--> Backend

Backend <--> PostgreSQL

Backend <--> Redis

Backend <--> Media

Backend <--> Celery

Backend --> WeasyPrint

Data model (ER overview):

mermaid

erDiagram

USER ||--o{ PATIENT : "one-to-one (patient profile)"

USER ||--o{ DOCTOR : "one-to-one (doctor profile)"

USER }o--o{ GROUP : "Django auth"

PATIENT ||--o{ APPOINTMENT : books

DOCTOR ||--o{ APPOINTMENT : attends

DOCTOR ||--o{ DOCTOR_AVAILABILITY : schedules

DOCTOR ||--o{ DOCTOR_LEAVE : leaves

DEPARTMENT ||--o{ DOCTOR : assigned_to

SERVICE ||--o{ APPOINTMENT : type

PATIENT ||--o{ ENCOUNTER : visits

ENCOUNTER ||--o{ PRESCRIPTION : creates

PRESCRIPTION ||--o{ PRESCRIPTION_ITEM : has

MEDICINE ||--o{ PRESCRIPTION_ITEM : references

LAB_TEST ||--o{ LAB_ORDER : defines

PATIENT ||--o{ LAB_ORDER : requests

DOCTOR ||--o{ LAB_ORDER : orders

LAB_ORDER ||--o{ LAB_RESULT : results

RADIOLOGY_TEST ||--o{ RADIOLOGY_ORDER : defines

RADIOLOGY_ORDER ||--o{ RADIOLOGY_RESULT : results

ROOM ||--o{ BED : has

PATIENT ||--o{ ADMISSION : admitted_to

ADMISSION ||--o{ BED_ALLOCATION : uses

BLOOD_UNIT ||--o{ BLOOD_ISSUE : issues

INSURANCE_COMPANY ||--o{ INSURANCE_POLICY : offers

PATIENT ||--o{ INSURANCE_POLICY : holds

INVOICE ||--o{ INVOICE_ITEM : contains

PATIENT ||--o{ INVOICE : billed

PAYMENT ||--|| INVOICE : settles

REFERRAL ||--|| PATIENT : for

REFERRAL ||--|| DOCTOR : by

REFERRAL ||--|| DEPARTMENT : to

NOTIFICATION }o--|| USER : "to user(s)"

AUDIT_LOG }o--|| USER : "actor"

OTP_TOKEN }o--|| USER : "verification"

Core Django apps:

- **core: utilities, constants (districts, provinces, blood groups), mixins, common models (Address, Contact, Attachment), audit logging, QR/Barcode helpers.**
- **accounts: CustomUser, roles/groups, auth (including Google via allauth), profiles, permissions, Jazzmin config.**
- **public: public pages, CMS-like content (services, departments, doctors list).**
- **patients: patient model, registration (email/mobile OTP), profile, card, QR/Barcode, dashboard.**
- **doctors: doctor profile, schedule, quota, leave, scanning.**
- **appointments: booking engine, availability checks, daily quotas, calendars.**
- **referrals: inter-department referrals.**

- **admissions: rooms, beds, admission, discharge, transfers.**
- **pharmacy: medicines, stock, dispensing, prescriptions.**
- **laboratory: test catalog, orders, samples, results, reports.**
- **radiology: modalities, orders, reports, images (attachment links).**
- **blood_bank: donors, screening, units, inventory, issues.**
- **ot: operation theater, procedures, bookings, notes.**
- **billing: invoices, items, tax, discounts, payment methods, receipts, PDFs.**
- **insurance: companies, policies, claims.**
- **medical_records: encounters, diagnoses, vitals, attachments, records management.**
- **notifications: in-app, email, SMS, optional real-time (Channels).**
- **reports: report definitions, data exports (CSV, XLSX, PDF).**
- **settings_app: hospital settings, departments, services, units, wards, integration keys.**
- **api: DRF endpoints for all modules.**
- **seed: management commands and fixtures.**

2) Step-by-Step Build Plan (80+ concrete steps)

1. **Create a new git repository and branch naming policy (e.g., feature/*, hotfix/*).**
2. **Create a Python 3.11 virtual environment; install Django 4.2 LTS.**
3. **Create project: django-admin startproject hamro_hospital .**
4. **Create apps: core, accounts, public, patients, doctors, appointments, referrals, admissions, pharmacy, laboratory, radiology, blood_bank, ot, billing, insurance, medical_records, notifications, reports, settings_app, api, seed.**
5. **Add django-environ; create .env with SECRET_KEY, DB creds, EMAIL, SMS keys, Google OAuth, eSewa.**
6. **Configure PostgreSQL connection, TIME_ZONE="Asia/Kathmandu", USE_TZ=True, LANGUAGE_CODE="en".**
7. **Install DRF, django-allauth, jazzmin, django-cors-headers, qrcode, python-barcode, WeasyPrint, celery, redis, django-axes.**
8. **Configure INSTALLED_APPS with Jazzmin first, then Django contrib apps, then project apps.**

9. **Configure AUTH_USER_MODEL = "accounts.User".**
10. **Setup staticfiles and media storage; local dev uses local media; prod uses S3-compatible.**
11. **Add Redis cache; set default cache backend to Redis; set session engine to cached_db.**
12. **Configure email backend (SMTP or transactional provider); SMS provider integration placeholders (e.g., Sparrow SMS, Twilio, or local gateway).**
13. **Add CORS headers for API if needed; restrict in production.**
14. **Configure security settings: CSRF_COOKIE_SECURE, SESSION_COOKIE_SECURE, SECURE_HSTS_SECONDS, SECURE_REFERRER_POLICY, X-Content-Type-Options, Content-Security-Policy.**
15. **Install Jazzmin; configure top menu, icons, login logo, brand colors to match the hospital logo theme.**
16. **Build custom User model (accounts.models.User) with fields: email (unique), phone, full_name, role default=None, is_patient, is_doctor flags.**
17. **Migrate initial schema (python manage.py makemigrations && migrate).**
18. **Create superuser.**
19. **Seed static data: provinces, 77 districts, blood groups, default departments, services, units, wards, rooms, beds (seed app commands).**
20. **Implement settings_app models: HospitalSettings (name, logo, address, primary_color, accent_color, theme options), Department, Service, Unit, Ward, Room, Bed.**
21. **Implement core models: Address, ContactInfo, Attachment, Choice models, AuditLog mixin (created_by, updated_by, timestamps, soft-delete), QR/Barcode utility functions.**
22. **Public site: base.html with Bootstrap 5.3, mobile-first responsive grid, offcanvas navbar.**
23. **Navbar: left logo + “Hamro Hospital” title; center menu (Home, About, Medical Services, Department, Doctors, Contact); right icons (notifications bell, Login button, Dark/Light toggle).**
24. **Implement dark mode toggle using Bootstrap data-bs-theme and localStorage; ensure it applies to dashboard too.**

25. **Homepage hero: slogan in Nepali “हाम्रो अस्पताल — स्वास्थ्य नै सबैभन्दा ठूलो धन हो।” with intro text, CTA cards.**
26. **Homepage cards: Medical Services, Departments, Doctors, Patient Registration, Hospital Extension Services — all clickable.**
27. **Services pages: /services -> list of services with cards; each detail page shows description, available doctors, working hours.**
28. **Departments UX: Use list of departments; clicking opens Bootstrap modal with description, room number, notes, contact info, and Close (X).**
29. **Doctors listing page: doctor cards with photo, name, department, unit; click to profile detail with weekly schedule, shifts, available time, leave status.**
30. **accounts: integrate django-allauth for Google login; configure OAuth callback; create Google “Patient” login path only; staff login remains username/password.**
31. **Patient registration: allow via Google, mobile number, or email; implement OTP verification via both Email and SMS.**
32. **OTP model (core or accounts): stores code, purpose, channel (email/mobile), hashed code, rate limiting, expiration (e.g., 10 min), throttling rules.**
33. **Upon verification, create patient account, generate Hospital ID (format: HH-YYYY-XXXXXX), QR code (PNG), Barcode (Code128).**
34. **Patients complete mandatory profile fields (collected as at hospital counter). Critical fields become locked after completion.**
35. **Enforce: only Registration Counter and Super Admin can edit locked patient fields. Track edits via AuditLog.**
36. **Patient dashboard: sidebar nav (Profile, Patient Card, Medical Records, Lab Reports, Prescriptions, Bills, Insurance, Admissions, Pharmacy, Book Appointment).**
37. **Show Patient Card printable view with QR and Barcode.**
38. **Implement doctor module: one-to-one with User; fields: department, unit, title, degrees, NMC number, contact, photo, bio.**
39. **Doctor schedule: DoctorAvailability with day_of_week, shift (morning/afternoon), start/end times, max daily quota. Allow doctor to edit only their own schedules via dashboard.**
40. **Doctor leave: DoctorLeave with start_date, end_date, reason; show prominent red “Doctor is currently on leave.” on profile and appointment pages when active.**

41. **Appointment engine: check doctor availability, daily quota, leave, and overlapping bookings. Appointment statuses: Requested, Confirmed, Checked-In, No-Show, Completed, Cancelled.**
42. **Doctor can search patients by name/phone/Hospital ID; can scan Barcode/QR via webcam (QuaggaJS/jsQR) in the browser.**
43. **Referral module: Doctor refer to Laboratory/Radiology/Pharmacy/Blood Bank/OT/Admission & Discharge; auto-create a referral record that appears in target department's dashboard queue.**
44. **Admissions: manage wards/rooms/beds; Admission record links patient to bed allocation and admission episodes (admit, transfer, discharge). Bed availability dashboard by ward.**
45. **Pharmacy: Medicine catalog, categories, strengths, forms, purchase price, sale price, tax, stock batches (batch_no, expiry), GRN entries, dispensing against prescription, pharmacy bills.**
46. **Laboratory: Test catalog (panels and tests), sample types, reference ranges; Lab Orders, Samples collection, Results entry, verification, result PDF with barcode/QR and signatures.**
47. **Radiology: Imaging order types (X-Ray, CT, MRI, USG), scheduling slot management, findings report, image attachments (DICOM optional via link).**
48. **Blood Bank: Donors, screening tests, blood groups, component separation, inventory, cross-match, issue to patients with traceability.**
49. **Operation Theater: Theaters, procedures, surgeon/assistant/anesthetist, OT booking schedule, notes, consumables usage.**
50. **Insurance: Companies, patient policies, pre-authorization requests, claims, statuses, EOB attachments.**
51. **Medical Records: Encounters, vitals, diagnoses (ICD-10 optional), procedures, attachments, discharge summaries.**
52. **Billing: Invoices, Items (with item_type: consultation, lab, radiology, bed, pharmacy, procedure), taxes, discounts, payment records.**
53. **Payments: cash (default), eSewa integration (redirect + webhook confirm), QR payment (Nepal QR or Fonepay QR; generate merchant QR string), store payment metadata.**
54. **Each bill PDF: hospital logo, bill number, patient name, phone, age, gender, barcode, payment method, date, time. Printable and downloadable with WeasyPrint.**

55. **Notifications: in-app (bell icon shows unread count), email alerts, optional SMS for key events (OTP, appointment confirmation, discharge). Persist to DB.**
56. **Dashboards by role with widgets: notifications, dark/light toggle, user profile, logout, recent activities + role-specific queues.**
57. **Super Admin dashboard modules: Departments, Doctors, Medicines, Hospital Services, Patients, Staff, Nursing, Admission & Discharge, OT, Blood Bank, Pharmacy, Laboratory, Radiology, Insurance, Accounts, Finance, Reports, Medical Records, Hospital Settings, Backup, Jazzmin Admin.**
58. **RBAC: map Django Groups to roles; assign permissions at model and viewset level; enforce object-level checks where needed (e.g., doctor editing only own schedule).**
59. **Global forms and validation strategy: server-side Django Forms/DRF serializers + client-side Bootstrap validation; strong constraints at DB level (unique, not null, foreign keys).**
60. **Error handling: custom error pages (400, 403, 404, 500), consistent API error format, logging via Django logging to file/console and Sentry-like system.**
61. **Audit logging: store create/update/delete with actor, timestamp, diff snapshot for sensitive models (patients, bills, medical records).**
62. **Data protection: protect PHI with least privilege access, encrypted storage for sensitive attachments, signed URLs if using S3.**
63. **Rate limit OTP, login attempts (django-axes), and sensitive endpoints to prevent abuse.**
64. **Add CSRF tokens on all forms; HTTP security headers via middleware.**
65. **Performance: database indexing, select_related/prefetch_related, Redis caching for read-mostly pages, pagination everywhere.**
66. **Reports: admin-friendly filters; export CSV/XLSX/PDF; common hospital KPIs: OPD volume, lab utilization, bed occupancy, average length of stay, pharmacy sales, revenue by department, claim statuses.**
67. **Backup: Nightly PostgreSQL pg_dump + media backup; Celery Beat triggers; store encrypted backups offsite; restoration runbooks.**
68. **Deployment: Dockerfiles for web, worker, beat; docker-compose for dev; staging and prod stacks with Nginx reverse proxy and Gunicorn; Let's Encrypt HTTPS; health checks.**
69. **Continuous Integration: pre-commit hooks (black, isort, flake8), unit tests, coverage; CI pipeline to run tests and security checks (bandit).**

70. **Logging/Monitoring: structured JSON logs, Prometheus/Grafana (optional), uptime probe.**
71. **Internationalization: enable i18n; store Nepali translations for UI labels; dynamic locale via user preference.**
72. **Accessibility: ARIA roles, proper color contrast, focus states, keyboard navigation, alt text for images.**
73. **Front-end polish: consistent margins, paddings, typography scales, iconography (Bootstrap Icons), tasteful micro-animations (fade, slide; avoid heavy transitions).**
74. **SEO for public pages; Open Graph tags; sitemap; robots.**
75. **Data import/export tools for migration; admin command to anonymize data for dev/test.**
76. **Doctor and Patient photo upload with automatic resize and compression; cache-busting URLs.**
77. **Barcode & QR consistency: Code128 for barcode; QR with hospital namespace "HH:" and patient hospital_id; render on Patient Card, Bills, OPD tickets, Prescriptions, Lab Reports, Pharmacy Bills.**
78. **Scanner UX: permission to camera, overlay guide, live detection with beeps; fallback manual entry.**
79. **Lab & Radiology result sign-offs: role-based signature steps and watermarks for "Preliminary" vs "Final".**
80. **Finance and accounts: chart of accounts (optional), link billing books; export for accounting systems.**
81. **Insurance-specific billing flows: separate invoice sequences, claim bundling, TPA attachments.**
82. **Caching strategy for public site and expensive dashboards; safe cache invalidation on model changes via signals.**
83. **Pagination and search with DRF; fuzzy search (optional) via pg_trgm extension.**
84. **Concurrency safety for quota/stock/beds with SELECT FOR UPDATE or database constraints.**
85. **Data dictionary docstrings on models and fields; auto-generate OpenAPI schema and API docs.**
86. **Training mode: seed demo users and fake data; onboarding tooltips for first login.**
87. **Feature flags for optional modules (Channels real-time, DICOM).**

88. **Go-live checklist: migrations, users/roles configured, payment keys live, SMS verified sender IDs, email domain verified, monitoring and backups tested.**
89. **Post-go-live support plan and patch-release process.**

3) Theming and Design System

Brand tokens (replace with real hex codes from your logo):

- **Primary (Blue): --hh-primary: #1E60A8**
- **Accent (Red, alerts only): --hh-accent: #D32F2F**
- **Neutral palette: slate/gray scales; success #198754; warning #FFC107; info #0DCAF0.**

SCSS token example (optional compile):

- **Variables override before importing Bootstrap.**

Example (scss):

```
/* theme/_variables.scss */
$primary: #1E60A8;
$danger: #D32F2F; // use sparingly for alerts/special actions
$body-color: #212529;
$border-radius: .5rem;
$enable-dark-mode: true;
```

Dark mode:

- **Use Bootstrap 5 color modes: <html data-bs-theme="light">.**
- **Toggle and remember via localStorage.**

JS snippet:

```
(function() {
const modeKey = 'hh-theme';
const root = document.documentElement;
const saved = localStorage.getItem(modeKey);
if (saved) root.setAttribute('data-bs-theme', saved);
document.getElementById('themeToggle')?.addEventListener('click', () => {
const cur = root.getAttribute('data-bs-theme') === 'dark' ? 'light' : 'dark';
root.setAttribute('data-bs-theme', cur);
localStorage.setItem(modeKey, cur);
});
})();
```

Navbar:

- **Left: Logo + “Hamro Hospital” text**
- **Middle: Home, About, Medical Services, Department, Doctors, Contact**
- **Right: Notification bell (public shows general notices), Login, Dark/Light toggle**

Responsive:

- **Collapse to offcanvas on mobile**
- **Touch targets ≥ 44px**
- **Tables responsive with .table-responsive**

4) Project Structure

hamro_hospital/

- **hamro_hospital/ (project settings)**
    - **settings/**
        - **base.py**
        - **dev.py**
        - **prod.py**
    - **urls.py**
    - **wsgi.py**
    - **asgi.py**
- **core/**
    - **models.py (Address, Attachment, AuditLog mixin)**
    - **utils.py (qr/barcode helpers, id generators)**
    - **choices.py (provinces, districts, blood groups)**
    - **middleware.py (audit, security headers)**
    - **admin.py**
    - **apps.py**
- **accounts/**
    - **models.py (User, Profile mixins)**
    - **auth_backends.py**
    - **forms.py (login, staff create)**

    - **adapters.py (allauth)**
    - **signals.py**
    - **urls.py/views.py/templates/**
- **public/**
    - **views.py/templates/public/ (home.html, services.html, departments.html, doctors.html)**
- **patients/**
    - **models.py (Patient, PatientCard)**
    - **views.py/forms.py/serializers.py/urls.py/templates/patients/**
    - **signals.py (generate QR/Barcode on create)**
- **doctors/**
    - **models.py (Doctor, Availability, Leave)**
    - **views.py/urls.py/templates/doctors/**
- **appointments/**
    - **models.py (Appointment, Slot)**
    - **services.py (availability checker)**
    - **views.py/serializers.py/templates/urls.py**
- **referrals/**
    - **models.py (Referral)**
- **admissions/**
    - **models.py (Ward, Room, Bed, Admission, BedAllocation, Transfer)**
- **pharmacy/**
    - **models.py (Medicine, StockBatch, Purchase, Prescription, PrescriptionItem, Dispense)**
- **laboratory/**
    - **models.py (LabTest, LabPanel, LabOrder, Sample, LabResult)**
- **radiology/**
    - **models.py (RadiologyTest, RadiologyOrder, RadiologyResult, Modality, ScheduleSlot)**

- **blood_bank/**
    - **models.py (Donor, Screening, BloodUnit, Inventory, CrossMatch, Issue)**
- **ot/**
    - **models.py (Theatre, Procedure, OTBooking, OTNote)**
- **billing/**
    - **models.py (Invoice, InvoiceItem, Payment)**
    - **pdf.py (WeasyPrint builders)**
- **insurance/**
    - **models.py (InsuranceCompany, Policy, Claim)**
- **medical_records/**
    - **models.py (Encounter, Vitals, Diagnosis, Procedure, Attachment)**
- **notifications/**
    - **models.py (Notification)**
- **reports/**
    - **builders.py (querysets, exports)**
- **settings_app/**
    - **models.py (HospitalSettings, Department, Service, Unit)**
- **api/**
    - **urls.py/viewsets.py/serializers.py/schemas.py**
- **seed/**
    - **management/commands/**
        - **seed_provinces_districts.py**
        - **seed_blood_groups.py**
        - **seed_departments_services.py**
        - **seed_rooms_beds.py**
        - **seed_doctors.py**
        - **seed_medicines.py**
        - **seed_lab_tests.py**

- **static/, media/, templates/, etc.**
- **docker/, compose files**
- **scripts/backup.sh, restore.sh**

5) Database Design (models, fields, constraints)

This section lists representative Django models. You can extend fields as needed, but this is production-ready structure.

accounts/models.py

- **Custom user with email-as-username design.**

```
from django.contrib.auth.models import AbstractUser, Group, Permission
from django.db import models

class User(AbstractUser):
username = None
email = models.EmailField(unique=True)
phone = models.CharField(max_length=20, unique=True, null=True, blank=True)
full_name = models.CharField(max_length=150, blank=True)
is_patient = models.BooleanField(default=False)
is_doctor = models.BooleanField(default=False)
# convenience role label (mirrors group) for UI
role = models.CharField(max_length=50, blank=True, null=True)
REQUIRED_FIELDS = []
USERNAME_FIELD = "email"
def str(self): return self.email
```

core/models.py

- **Common mixins and helpers.**

```
from django.db import models
from django.conf import settings

class TimeStampedModel(models.Model):
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
class Meta: abstract = True

class AuditModel(TimeStampedModel):
created_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True,
on_delete=models.SET_NULL, related_name="+")
updated_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True,
on_delete=models.SET_NULL, related_name="+")
```

is_active = models.BooleanField(default=True)
class Meta: abstract = True

class Address(models.Model):
province = models.CharField(max_length=50)
district = models.CharField(max_length=50)
municipality = models.CharField(max_length=100, blank=True)
ward_no = models.CharField(max_length=10, blank=True)
street = models.CharField(max_length=150, blank=True)

class Attachment(AuditModel):
file = models.FileField(upload_to="attachments/%Y/%m/")
category = models.CharField(max_length=50, blank=True)
description = models.TextField(blank=True)

OTP tokens (for email/mobile):

class OtpToken(TimeStampedModel):
user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="otp_tokens", null=True, blank=True)
channel = models.CharField(max_length=10, choices=[("email","Email"),("sms","SMS")])
destination = models.CharField(max_length=100)
purpose = models.CharField(max_length=50) # "registration","login","verify_contact"
code_hash = models.CharField(max_length=128)
expires_at = models.DateTimeField()
attempts = models.PositiveIntegerField(default=0)
is_used = models.BooleanField(default=False)

settings_app/models.py

class HospitalSettings(AuditModel):
name = models.CharField(max_length=150)
logo = models.ImageField(upload_to="branding/", null=True, blank=True)
primary_color = models.CharField(max_length=7, default="#1E60A8")
accent_color = models.CharField(max_length=7, default="#D32F2F")
phone = models.CharField(max_length=20, blank=True)
email = models.EmailField(blank=True)
address = models.OneToOneField("core.Address", null=True, blank=True,
on_delete=models.SET_NULL)

class Department(AuditModel):
name = models.CharField(max_length=100, unique=True)
description = models.TextField(blank=True)
room_number = models.CharField(max_length=20, blank=True)

notes = models.TextField(blank=True)
contact_phone = models.CharField(max_length=20, blank=True)
contact_email = models.EmailField(blank=True)

class Service(AuditModel):
name = models.CharField(max_length=100, unique=True)
slug = models.SlugField(unique=True)
description = models.TextField(blank=True)
working_hours = models.CharField(max_length=100, blank=True)

class Unit(AuditModel):
name = models.CharField(max_length=100, unique=True)
department = models.ForeignKey(Department, on_delete=models.CASCADE,
related_name="units")

admissions/models.py

class Ward(AuditModel):
name = models.CharField(max_length=100, unique=True)

class Room(AuditModel):
ward = models.ForeignKey(Ward, on_delete=models.CASCADE, related_name="rooms")
number = models.CharField(max_length=20)

class Bed(AuditModel):
room = models.ForeignKey(Room, on_delete=models.CASCADE, related_name="beds")
bed_no = models.CharField(max_length=20)
is_occupied = models.BooleanField(default=False)

class Admission(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="admissions")
admit_time = models.DateTimeField()
discharge_time = models.DateTimeField(null=True, blank=True)
reason = models.TextField(blank=True)
current_bed = models.ForeignKey(Bed, on_delete=models.SET_NULL, null=True,
blank=True)

class BedAllocation(AuditModel):
admission = models.ForeignKey(Admission, on_delete=models.CASCADE,
related_name="allocations")
bed = models.ForeignKey(Bed, on_delete=models.CASCADE)
start_time = models.DateTimeField()
end_time = models.DateTimeField(null=True, blank=True)

patients/models.py

```
from django.db import models
from django.conf import settings

class Patient(AuditModel):
user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="patient_profile")
hospital_id = models.CharField(max_length=20, unique=True) # HH-YYYY-XXXXXX
date_of_birth = models.DateField(null=True, blank=True)
gender = models.CharField(max_length=10,
choices=[("M","Male"),("F","Female"),("O","Other")], blank=True)
blood_group = models.CharField(max_length=5, blank=True)
phone = models.CharField(max_length=20, blank=True)
email = models.EmailField(blank=True)
address = models.OneToOneField("core.Address", null=True, blank=True,
on_delete=models.SET_NULL)
qr_image = models.ImageField(upload_to="qr/", blank=True)
barcode_image = models.ImageField(upload_to="barcode/", blank=True)
is_profile_complete = models.BooleanField(default=False)
locked_fields = models.JSONField(default=list, blank=True) # list of field names locked
after verification

class PatientCard(AuditModel):
patient = models.OneToOneField(Patient, on_delete=models.CASCADE,
related_name="card")
issued_at = models.DateTimeField(auto_now_add=True)
notes = models.TextField(blank=True)

doctors/models.py

class Doctor(AuditModel):
user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="doctor_profile")
department = models.ForeignKey("settings_app.Department",
on_delete=models.SET_NULL, null=True, related_name="doctors")
unit = models.ForeignKey("settings_app.Unit", on_delete=models.SET_NULL, null=True,
blank=True)
title = models.CharField(max_length=50, blank=True) # Dr., Prof., etc.
degrees = models.CharField(max_length=200, blank=True)
nmc_no = models.CharField(max_length=50, blank=True)
photo = models.ImageField(upload_to="doctors/", blank=True)
bio = models.TextField(blank=True)
daily_quota = models.PositiveIntegerField(default=40)
```

```
class DoctorAvailability(AuditModel):
doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE,
related_name="availabilities")
day_of_week =
models.IntegerField(choices=[(0,"Mon"),(1,"Tue"),(2,"Wed"),(3,"Thu"),(4,"Fri"),(5,"Sat"),(
6,"Sun")])
shift = models.CharField(max_length=20,
choices=[("morning","Morning"),("afternoon","Afternoon")])
start_time = models.TimeField()
end_time = models.TimeField()
max_quota = models.PositiveIntegerField(default=40)

class DoctorLeave(AuditModel):
doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE, related_name="leaves")
start_date = models.DateField()
end_date = models.DateField()
reason = models.TextField(blank=True)

appointments/models.py

class Appointment(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="appointments")
doctor = models.ForeignKey("doctors.Doctor", on_delete=models.CASCADE,
related_name="appointments")
service = models.ForeignKey("settings_app.Service", on_delete=models.SET_NULL,
null=True, blank=True)
date = models.DateField()
time = models.TimeField()
status = models.CharField(max_length=20, choices=[
("requested","Requested"),("confirmed","Confirmed"),("checked_in","Checked-In"),
("completed","Completed"),("no_show","No-Show"),("cancelled","Cancelled")
], default="requested")
notes = models.TextField(blank=True)
ticket_no = models.CharField(max_length=20, blank=True)

referrals/models.py

class Referral(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="referrals")
referred_by = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL,
null=True, related_name="referrals_made")
```

```
to_department = models.ForeignKey("settings_app.Department",
on_delete=models.SET_NULL, null=True, related_name="referrals_received")
reason = models.TextField()
status = models.CharField(max_length=20, choices=[("new","New"),("in_progress","In
Progress"),("completed","Completed")], default="new")
```

pharmacy/models.py

```
class Medicine(AuditModel):
name = models.CharField(max_length=150)
generic_name = models.CharField(max_length=150, blank=True)
form = models.CharField(max_length=50, blank=True) # tablet, syrup, injection
strength = models.CharField(max_length=50, blank=True)
hsn_code = models.CharField(max_length=20, blank=True)
tax_percent = models.DecimalField(max_digits=5, decimal_places=2, default=0)
purchase_price = models.DecimalField(max_digits=10, decimal_places=2, default=0)
sale_price = models.DecimalField(max_digits=10, decimal_places=2, default=0)
is_prescription_required = models.BooleanField(default=True)

class StockBatch(AuditModel):
medicine = models.ForeignKey(Medicine, on_delete=models.CASCADE,
related_name="batches")
batch_no = models.CharField(max_length=50)
expiry_date = models.DateField()
quantity = models.PositiveIntegerField()
quantity_available = models.PositiveIntegerField()

class Prescription(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="prescriptions")
doctor = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL, null=True,
related_name="prescriptions")
encounter = models.ForeignKey("medical_records.Encounter",
on_delete=models.SET_NULL, null=True, blank=True)
notes = models.TextField(blank=True)

class PrescriptionItem(AuditModel):
prescription = models.ForeignKey(Prescription, on_delete=models.CASCADE,
related_name="items")
medicine = models.ForeignKey(Medicine, on_delete=models.CASCADE)
dose = models.CharField(max_length=50)
frequency = models.CharField(max_length=50)
```

duration = models.CharField(max_length=50)
quantity = models.PositiveIntegerField()

class Dispense(AuditModel):
prescription = models.ForeignKey(Prescription, on_delete=models.CASCADE,
related_name="dispenses")
items = models.ManyToManyField(PrescriptionItem, through="DispenseItem")
invoice = models.ForeignKey("billing.Invoice", on_delete=models.SET_NULL, null=True,
blank=True)

class DispenseItem(models.Model):
dispense = models.ForeignKey(Dispense, on_delete=models.CASCADE)
prescription_item = models.ForeignKey(PrescriptionItem, on_delete=models.CASCADE)
batch = models.ForeignKey(StockBatch, on_delete=models.SET_NULL, null=True)
quantity = models.PositiveIntegerField()

laboratory/models.py

class LabTest(AuditModel):
code = models.CharField(max_length=20, unique=True)
name = models.CharField(max_length=150)
sample_type = models.CharField(max_length=50)
method = models.CharField(max_length=100, blank=True)
reference_range = models.CharField(max_length=200, blank=True)
price = models.DecimalField(max_digits=10, decimal_places=2, default=0)

class LabPanel(AuditModel):
name = models.CharField(max_length=150)
tests = models.ManyToManyField(LabTest, related_name="panels")

class LabOrder(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="lab_orders")
ordered_by = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL,
null=True)
tests = models.ManyToManyField(LabTest, related_name="orders")
status = models.CharField(max_length=20,
choices=[("new","New"),("collected","Collected"),("processing","Processing"),("verified",
"Verified")], default="new")
invoice = models.ForeignKey("billing.Invoice", on_delete=models.SET_NULL, null=True,
blank=True)

class Sample(AuditModel):
order = models.ForeignKey(LabOrder, on_delete=models.CASCADE,

```
related_name="samples")
sample_code = models.CharField(max_length=30)
collected_at = models.DateTimeField(null=True, blank=True)

class LabResult(AuditModel):
order = models.OneToOneField(LabOrder, on_delete=models.CASCADE,
related_name="result")
result_json = models.JSONField(default=dict) # key:value pairs for tests
verified_by = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL,
null=True, blank=True, related_name="lab_results_verified")
verified_at = models.DateTimeField(null=True, blank=True)

radiology/models.py

class RadiologyTest(AuditModel):
code = models.CharField(max_length=20, unique=True)
name = models.CharField(max_length=150)
modality = models.CharField(max_length=50) # X-Ray, CT, MRI, USG
price = models.DecimalField(max_digits=10, decimal_places=2, default=0)

class RadiologyOrder(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="radiology_orders")
ordered_by = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL,
null=True)
test = models.ForeignKey(RadiologyTest, on_delete=models.CASCADE)
schedule_time = models.DateTimeField(null=True, blank=True)
status = models.CharField(max_length=20,
choices=[("new","New"),("scheduled","Scheduled"),("completed","Completed"),("reporte
d","Reported")], default="new")
invoice = models.ForeignKey("billing.Invoice", on_delete=models.SET_NULL, null=True,
blank=True)

class RadiologyResult(AuditModel):
order = models.OneToOneField(RadiologyOrder, on_delete=models.CASCADE,
related_name="result")
findings = models.TextField()
impression = models.TextField()
attachments = models.ManyToManyField("core.Attachment", blank=True)
reported_by = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL,
null=True, related_name="radiology_reports")

blood_bank/models.py
```

```
class Donor(AuditModel):
name = models.CharField(max_length=150)
phone = models.CharField(max_length=20, blank=True)
blood_group = models.CharField(max_length=5)
last_donation = models.DateField(null=True, blank=True)

class BloodUnit(AuditModel):
donor = models.ForeignKey(Donor, on_delete=models.SET_NULL, null=True)
group = models.CharField(max_length=5)
component = models.CharField(max_length=20) # WB, PRBC, FFP, PLT
unit_no = models.CharField(max_length=30, unique=True)
expiry_date = models.DateField()
is_available = models.BooleanField(default=True)

class Issue(AuditModel):
unit = models.ForeignKey(BloodUnit, on_delete=models.CASCADE)
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE)
issued_at = models.DateTimeField(auto_now_add=True)
cross_match_ok = models.BooleanField(default=False)

ot/models.py

class Theatre(AuditModel):
name = models.CharField(max_length=100)
location = models.CharField(max_length=100, blank=True)

class Procedure(AuditModel):
code = models.CharField(max_length=20, unique=True)
name = models.CharField(max_length=150)
price = models.DecimalField(max_digits=10, decimal_places=2, default=0)

class OTBooking(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE)
doctor = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL, null=True)
procedure = models.ForeignKey(Procedure, on_delete=models.SET_NULL, null=True)
theatre = models.ForeignKey(Theatre, on_delete=models.SET_NULL, null=True)
schedule_time = models.DateTimeField()
status = models.CharField(max_length=20,
choices=[("scheduled","Scheduled"),("in_progress","In
Progress"),("completed","Completed"),("cancelled","Cancelled")], default="scheduled")

billing/models.py

class Invoice(AuditModel):
invoice_no = models.CharField(max_length=30, unique=True)
```

patient = models.ForeignKey("patients.Patient", on_delete=models.SET_NULL, null=True, related_name="invoices")
department = models.ForeignKey("settings_app.Department", on_delete=models.SET_NULL, null=True, blank=True)
status = models.CharField(max_length=20, choices=[("unpaid","Unpaid"),("partial","Partial"),("paid","Paid"),("void","Void")], default="unpaid")
total = models.DecimalField(max_digits=12, decimal_places=2, default=0)
tax = models.DecimalField(max_digits=12, decimal_places=2, default=0)
discount = models.DecimalField(max_digits=12, decimal_places=2, default=0)
grand_total = models.DecimalField(max_digits=12, decimal_places=2, default=0)
paid_amount = models.DecimalField(max_digits=12, decimal_places=2, default=0)
due_amount = models.DecimalField(max_digits=12, decimal_places=2, default=0)
payment_method = models.CharField(max_length=20, blank=True) # last method
metadata = models.JSONField(default=dict, blank=True)

class InvoiceItem(AuditModel):
invoice = models.ForeignKey(Invoice, on_delete=models.CASCADE, related_name="items")
item_type = models.CharField(max_length=30) # consultation, lab, radiology, bed, medicine, procedure
description = models.CharField(max_length=200)
quantity = models.DecimalField(max_digits=10, decimal_places=2, default=1)
unit_price = models.DecimalField(max_digits=12, decimal_places=2)
amount = models.DecimalField(max_digits=12, decimal_places=2)

class Payment(AuditModel):
invoice = models.ForeignKey(Invoice, on_delete=models.CASCADE, related_name="payments")
method = models.CharField(max_length=20, choices=[("cash","Cash"),("esewa","eSewa"),("qr","QR")])
txn_id = models.CharField(max_length=100, blank=True)
amount = models.DecimalField(max_digits=12, decimal_places=2)
paid_at = models.DateTimeField(auto_now_add=True)
metadata = models.JSONField(default=dict, blank=True)

insurance/models.py

class InsuranceCompany(AuditModel):
name = models.CharField(max_length=150, unique=True)
contact_email = models.EmailField(blank=True)
phone = models.CharField(max_length=20, blank=True)

```
class Policy(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="policies")
company = models.ForeignKey(InsuranceCompany, on_delete=models.CASCADE,
related_name="policies")
policy_no = models.CharField(max_length=50)
valid_from = models.DateField()
valid_to = models.DateField()

class Claim(AuditModel):
policy = models.ForeignKey(Policy, on_delete=models.CASCADE, related_name="claims")
invoice = models.ForeignKey("billing.Invoice", on_delete=models.CASCADE,
related_name="claims")
status = models.CharField(max_length=20,
choices=[("submitted","Submitted"),("approved","Approved"),("rejected","Rejected"),("p
aid","Paid")], default="submitted")
attachments = models.ManyToManyField("core.Attachment", blank=True)
remarks = models.TextField(blank=True)

medical_records/models.py

class Encounter(AuditModel):
patient = models.ForeignKey("patients.Patient", on_delete=models.CASCADE,
related_name="encounters")
doctor = models.ForeignKey("doctors.Doctor", on_delete=models.SET_NULL, null=True,
related_name="encounters")
date = models.DateField(auto_now_add=True)
notes = models.TextField(blank=True)

class Vitals(AuditModel):
encounter = models.OneToOneField(Encounter, on_delete=models.CASCADE,
related_name="vitals")
height_cm = models.DecimalField(max_digits=5, decimal_places=2, null=True, blank=True)
weight_kg = models.DecimalField(max_digits=5, decimal_places=2, null=True, blank=True)
bp = models.CharField(max_length=20, blank=True)
pulse = models.IntegerField(null=True, blank=True)
temperature_c = models.DecimalField(max_digits=4, decimal_places=1, null=True,
blank=True)

class Diagnosis(AuditModel):
encounter = models.ForeignKey(Encounter, on_delete=models.CASCADE,
related_name="diagnoses")
```

code = models.CharField(max_length=20, blank=True) # ICD-10 optional
description = models.CharField(max_length=200)

notifications/models.py

class Notification(AuditModel):
user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name="notifications")
title = models.CharField(max_length=150)
body = models.TextField(blank=True)
url = models.CharField(max_length=300, blank=True)
is_read = models.BooleanField(default=False)

Constraints and indexes:

- **Unique: email, hospital_id, invoice_no, blood unit_no, lab/radiology codes.**
- **Indexes: foreign keys by default; add composite indexes for appointment (doctor,date), bed occupancy (room,bed_no), stock (medicine,batch_no).**
- **Data integrity via not null, on_delete behaviors, server-level constraints for availability and quotas (enforced in services.py with transaction.atomic and select_for_update for counters).**

6) Authentication, Permissions, and Roles

Custom User:

- **Email as username**
- **Flags: is_patient, is_doctor**
- **Separate Patient and Doctor profiles (one-to-one)**

Django Groups (roles):

- **super_admin**
- **doctor**
- **nurse**
- **registration_counter**
- **pharmacy**
- **laboratory**
- **radiology**
- **blood_bank**

- **operation_theater**
- **admission_discharge**
- **cash_counter**
- **accounts**
- **insurance**
- **medical_records**
- **patient**

Permission mapping (illustrative):

- **patient: view_own_profile, view_own_records, book_appointment, view_own_invoices**
- **doctor: view_patients_assigned, update_own_profile, manage_own_schedule, set_leave, refer_patient, view_medical_history, create_prescription, view_lab_radiology_results**
- **registration_counter: create_patient, update_patient_locked_fields, issue_patient_card**
- **pharmacy: manage_medicines, dispense, manage_stock**
- **laboratory: manage_lab_orders, enter_results, verify_results**
- **radiology: manage_radiology_orders, create_reports**
- **blood_bank: manage_blood_inventory, issue_units**
- **operation_theater: schedule_ot, manage_procedures**
- **admission_discharge: admit, transfer, discharge, manage_beds**
- **cash_counter/accounts: create_invoice, take_payment, refunds, finance_reports**
- **insurance: manage_policies, submit_claims**
- **medical_records: manage_encounters, finalize_documents**
- **super_admin: all permissions**

Object-level checks:

- **Patient critical fields editable only by registration_counter or super_admin.**
- **Doctor can only update their own schedule/leave/quota.**
- **Role-specific dashboards: restrict list/querysets to allowed scope.**

Two-factor/OTP:

- **OTP for patient registration and optional for staff login. Rate-limit OTP generation attempts per IP and per destination.**

7) Public Website Implementation

URLs:

- **/ -> Home**
- **/about -> About**
- **/services -> Services list**
- **/services/<slug> -> Service detail**
- **/departments -> Departments list with click-to-modal**
- **/doctors -> Doctors list**
- **/doctors/<id-or-slug> -> Doctor profile**
- **/contact -> Contact**
- **/login -> Login**
- **/register -> Patient registration (Google/Mobile/Email)**

Homepage:

- **Hero with slogan in Nepali and intro**
- **Cards for Services, Departments, Doctors, Patient Registration, Extension Services**
- **All clickable; sleek hover animations (light, no bounce)**

Departments modal content:

- **Description, room number, notes, contact phone/email, Close (X)**

Doctors profile:

- **Photo, name, department, unit, degrees**
- **Weekly schedule table (Mon–Sun), shifts morning/afternoon**
- **Available time slots; "Doctor is currently on leave" in red when active leave**

Dark mode toggle:

- **Appears in navbar; persists via localStorage**
- **Applies across dashboard via shared base template**

8) Patient Registration and OTP Flows

Paths:

- **Register via Google: /accounts/google/login/ (allauth)**
- **Register via Email: enter email -> send OTP -> verify -> create user -> complete profile**
- **Register via Mobile: enter mobile (+977 format) -> send OTP via SMS -> verify -> create user -> complete profile**

OTP flow:

1. **User enters destination (email/phone) and purpose ("registration").**
2. **Server generates random 6-digit code; stores hash + expiry + throttling meta.**
3. **Send code via email/SMS; show "enter OTP" screen.**
4. **Verify; mark token used; create user if new; assign is_patient=True; create Patient; generate hospital_id; generate QR and barcode images (signals).**
5. **Force profile completion form; lock critical fields after submission.**

Security:

- **Rate limit OTP to 3 per hour per destination; 6 invalid attempts lock for 30 minutes.**
- **Store only hashed codes; do not log raw OTP.**
- **Expire OTP in 10 minutes.**

9) Doctor Module and Scheduling

- **Super Admin creates doctor user + temp password; doctor resets password on first login.**
- **Doctor dashboard widgets: Today's appointments, pending referrals, quick actions (Set Leave, Update Schedule, Set Quota).**
- **Availability UI: weekly grid; drag handles or time pickers for start/end, per-shift quota.**
- **Leave UI: date range picker; overlaps blocked; warning if existing appointments exist (force notify and reschedule).**
- **On leave banner: red alert on doctor profile/public page; appointment booking disabled for leave dates.**

Scanning:

- **Button “Scan QR/Barcode” opens modal with camera preview**
- **Use jsQR (QR) and QuaggaJS (barcode) to decode**
- **On decode -> fetch patient via API -> show quick patient summary**

10) Appointments and Workflows

Booking rules:

- **Only book in available shifts**
- **Respect doctor daily quota (availability.max_quota or doctor.daily_quota)**
- **Skip dates within any DoctorLeave**
- **Status transitions: requested -> confirmed -> checked_in -> completed or cancelled/no_show**
- **Patients can book online for OPD; registration counter can book on behalf**

Ticketing:

- **Generate ticket_no per appointment (e.g., A-YYYYMMDD-### by department/doctor)**
- **Print small OPD ticket with barcode**

Notifications:

- **Email/SMS/app notification on booking and changes**
- **Reminders 24h and 2h before (Celery Beat)**

11) Departments, Services, and Referrals

- **Services replace old “Departments” in public site as main service catalog**
- **Departments exist internally (OT, Lab, Radiology, X-Ray, CT, MRI, Nursing, Pharmacy) for workflows, staff, billing, and referrals**
- **Doctor referral: creates Referral entry -> appears in target department queue**
- **Departments dashboard: filter by “new/in_progress/completed”, click row to process**

12) Admissions, Beds, and Discharge

- **Bed board: color-coded statuses (available, occupied, cleaning, reserved)**
- **Admission create: select available bed -> allocate -> create invoice items (admission fee; daily bed charges via nightly job)**

- **Transfer: close previous BedAllocation, open new; update invoices**
- **Discharge: set discharge_time; finalize bed charges; print discharge summary; notify billing**

13) Laboratory Module

- **Test catalog with codes, names, sample types, prices**
- **Lab order from doctor referral or direct OPD: attach to invoice**
- **Sample collection: generate sample code with barcode; print label**
- **Result entry UI: dynamic fields based on tests; reference ranges; flags for abnormal values**
- **Verification: two-step if required (tech -> pathologist)**
- **PDF: header with logo; patient info; barcode/QR; tests table; ranges; sign-off; watermark if "preliminary"**

14) Radiology Module

- **Order scheduling by modality; slot calendar**
- **Result report: findings, impression, attachments (images or links)**
- **PDF: similar to lab; include images/thumbnails if appropriate**
- **DICOM integration optional via external PACS links (store URL/reference only)**

15) Pharmacy and Medicines

- **Medicine catalog with strength, form, prices, tax**
- **Stock batches with expiry; FIFO dispensing**
- **Prescription builder in doctor UI; patient sees prescriptions in dashboard**
- **Dispense flow generates invoice items automatically; decrement stock**
- **Expiry alerts; low-stock alerts; stock audit logs**
- **Seed 500+ medicines (see Seed section)**

16) Blood Bank

- **Donor registry; eligibility tracking**
- **Unit creation after donation; blood grouping; screening**
- **Inventory by group and component; alerts on low stock or near expiry**
- **Issue to patients with cross-match status; record linkage to encounter/invoice**

17) OT (Operation Theater)

- **Procedure catalog; OT scheduling; resource conflict checks**
- **Pre-op checklist; post-op notes; consumables costing into invoice items**
- **Status board (scheduled, in progress, completed, cancelled)**

18) Insurance

- **Companies and policies linked to patients**
- **Pre-auth requests with attachments**
- **Claims linked to invoices; statuses and remarks**
- **Reports: claim aging, rejection reasons**

19) Billing, Payments, PDF, and QR/eSewa

Invoice number generation:

- **Sequential per day or hospital-wide (HH-INV-YYYYMM-XXXX)**

Payment methods:

- **Cash: mark paid immediately**
- **eSewa:**
    - **Create payment request; redirect or open in modal**
    - **On webhook callback: verify signature; update invoice->paid**
    - **Store txn_id and raw payload in Payment.metadata**
- **QR Payment:**
    - **Generate merchant QR code (per invoice or static merchant QR)**
    - **User scans, pays, staff confirms receipt via PSP portal or webhook if available**
    - **Update payment accordingly**

PDF (WeasyPrint):

- **invoice_pdf(request, invoice_id) -> returns PDF**
- **Header with logo; body shows line items, totals, taxes, discounts**
- **Footer with terms, QR linking to invoice verification URL**
- **Include Barcode (Code128) of invoice_no**

Barcodes and QR:

- **Use qrcode to generate PNG; python-barcode for Code128**
- **Store on disk and link; optionally generate on-the-fly for PDFs**

20) Dashboards and UI

All dashboards share:

- **Topbar: notifications bell, theme toggle, profile, logout**
- **Left sidebar: role-specific menu**
- **Main widgets: KPIs, quick actions, recent activity**

Role specifics:

- **Super Admin: everything + system settings + backups + Jazzmin admin**
- **Doctor: today's schedule, queue, referrals, set schedule/leave/quota, patient search/scan**
- **Nurse: vitals entry, ward boards, tasks**
- **Registration Counter: create/edit patients, issue cards, appointments**
- **Pharmacy: dispensing queue, stock dashboard**
- **Laboratory: orders queue by status, result entry, verification**
- **Radiology: schedule, reporting, attachments**
- **Blood Bank: inventory dashboard, donor schedules**
- **OT: OT calendar, resources, notes**
- **Admission & Discharge: bed board, admission/transfer/discharge**
- **Cash Counter/Accounts: invoices, payments, daily close, finance reports**
- **Insurance: policies, claims, aging**
- **Medical Records: encounters, document management**
- **Patient: profile, card, records, lab/radiology, prescriptions, bills, insurance, admissions, appointments**

21) Views, URLs, Forms, Templates (patterns)

- **Use class-based views and DRF viewsets.**
- **Use template inheritance: base.html -> dashboard_base.html and public_base.html**
- **Use crispy-forms or Bootstrap 5 form helpers for clean layouts.**

Example URLs:

- **public/urls.py: home, about, services, service_detail, departments, doctors, doctor_detail, contact, login, register**
- **patients/urls.py: profile, card, records, lab, prescriptions, bills, insurance, admissions, book_appointment**
- **doctors/urls.py: dashboard, schedule, leave, quota, referrals**
- **appointments/urls.py: create, list, confirm, cancel**
- **billing/urls.py: invoice list/detail/pdf, payment init/callback**
- **api/urls.py: DRF routers for each model**

Example DRF Router:

```
from rest_framework import routers
from .viewsets import PatientViewSet, DoctorViewSet, AppointmentViewSet,
InvoiceViewSet
router = routers.DefaultRouter()
router.register(r'patients', PatientViewSet, basename='patients')
router.register(r'doctors', DoctorViewSet, basename='doctors')
router.register(r'appointments', AppointmentViewSet, basename='appointments')
router.register(r'invoices', InvoiceViewSet, basename='invoices')
```

22) APIs (selected endpoints)

Auth:

- **POST /api/auth/login**
- **POST /api/auth/logout**
- **POST /api/auth/otp/request {channel, destination, purpose}**
- **POST /api/auth/otp/verify {destination, code, purpose}**

Patients:

- **GET /api/patients/me**
- **PATCH /api/patients/me/profile (limited fields)**
- **GET /api/patients/{id}**
- **GET /api/patients/search?q=**
- **GET /api/patients/{id}/card**
- **GET /api/patients/{id}/barcode**

- **GET /api/patients/{id}/qrcode**

Doctors:

- **GET /api/doctors**
- **GET /api/doctors/{id}**
- **GET /api/doctors/{id}/schedule**
- **POST /api/doctors/me/schedule**
- **POST /api/doctors/me/leave**
- **POST /api/doctors/me/quota**

Appointments:

- **POST /api/appointments (patient_id, doctor_id, service_id, date, time)**
- **GET /api/appointments?doctor_id=&date=**
- **PATCH /api/appointments/{id} (status)**
- **GET /api/availability?doctor_id=&date=**

Referrals:

- **POST /api/referrals**
- **GET /api/referrals?department_id=&status=**

Admissions:

- **GET /api/beds/board**
- **POST /api/admissions**
- **POST /api/admissions/{id}/transfer**
- **POST /api/admissions/{id}/discharge**

Pharmacy:

- **GET /api/medicines?search=**
- **POST /api/dispense**
- **GET /api/stock/alerts**

Lab:

- **POST /api/lab/orders**
- **GET /api/lab/orders?status=**

- **POST /api/lab/orders/{id}/collect**
- **POST /api/lab/orders/{id}/result**
- **GET /api/lab/orders/{id}/pdf**

Radiology:

- **POST /api/radiology/orders**
- **POST /api/radiology/orders/{id}/report**
- **GET /api/radiology/orders/{id}/pdf**

Billing:

- **POST /api/invoices**
- **POST /api/invoices/{id}/items**
- **POST /api/payments (method, invoice_id)**
- **POST /api/payments/esewa/callback**

Notifications:

- **GET /api/notifications**
- **POST /api/notifications/{id}/read**

23) Business Rules and Validation

- **Patient critical info (name, dob, gender, id, primary phone) locked after verification; only Registration Counter or Super Admin edits.**
- **Doctor leave blocks appointment booking; admin can override with forced note.**
- **Pharmacy cannot dispense more than available stock; if insufficient, split or backorder.**
- **Lab/Radiology cannot mark "verified/reported" without required fields; add digital signature (name + timestamp).**
- **Bed allocation: cannot overlap times; enforce with database constraints and transaction locks.**
- **Billing totals recalc on every mutation; due_amount must never be negative.**
- **OTP attempts throttled; after 6 failed attempts, lock OTP for 30 min.**
- **Phone numbers validated for +977; emails RFC-compliant.**

24) Error Handling and UX

- **Show inline errors for form fields; highlight invalid inputs with Bootstrap is-invalid.**
- **Display toast notifications for success/error.**
- **404 page for missing routes; 403 for unauthorized; 500 generic error with tracking ID (log correlation ID).**
- **Graceful degradation for camera/scanner: show manual entry fallback.**

25) Security Hardening

- **HTTPS everywhere; HSTS enabled.**
- **Secure cookies; CSRF on all forms and API where session-based.**
- **Rate limiting: login, OTP, payment attempts.**
- **Input sanitization; use Django auto-escaping. Sanitize file uploads and restrict types.**
- **Access controls enforced on server; do not rely on UI-only checks.**
- **Secret management via environment variables; never commit secrets.**
- **Regular dependency scanning; apply security patches promptly.**
- **Audit logs for patient data edits, billing, results, and discharge changes.**

26) Reporting

- **Built-in reports (filters + export):**
    - **OPD volume by day/doctor/department**
    - **Lab utilization by test/panel/date**
    - **Bed occupancy and average length of stay**
    - **Pharmacy sales by category/medicine/date**
    - **Revenue by department and payment method**
    - **Insurance claims status and aging**
    - **Doctor workload and wait times**
- **Export to CSV, XLSX, and PDF**
- **Schedule summary emails to admins (Celery Beat)**

27) Notifications

- **Notification model for in-app alerts**
- **Email via SMTP provider**

- **SMS via gateway (configure provider during deployment)**
- **Optional real-time via Django Channels: subscribe per-user; fallback to polling**
- **Patients get reminders; staff get queue updates (referrals, orders, admissions)**

28) Backup and Disaster Recovery

- **Nightly pg_dump with timestamp to secure storage (S3 or offsite server)**
- **Media backup (rsync to cold storage or S3)**
- **Encrypt backups (e.g., age/gpg)**
- **Weekly test restore to staging**
- **scripts/backup.sh and scripts/restore.sh**
- **Document RPO/RTO targets**

29) Deployment

- **Dockerfile for web (Django + Gunicorn), worker (Celery), beat (Celery Beat)**
- **docker-compose for dev; k8s/compose for prod**
- **Nginx reverse proxy; Let's Encrypt SSL**
- **collectstatic to serve via Nginx**
- **Health checks on /healthz**
- **Rolling updates; migrations with zero-downtime patterns where possible**
- **Logging to stdout in JSON; aggregated by your log system**

30) Sample Data and Seeders

Management commands in seed app to populate the system. Use Faker for realistic data where appropriate.

Provinces (7) and 77 Districts (Koshi, Madhesh, Bagmati, Gandaki, Lumbini, Karnali, Sudurpashchim):

- **Provinces:**
    - **Koshi**
    - **Madhesh**
    - **Bagmati**
    - **Gandaki**
    - **Lumbini**

    - **Karnali**
    - **Sudurpashchim**
- **Districts by Province:**
    - **Koshi: Bhojpur, Dhankuta, Ilam, Jhapa, Khotang, Morang, Okhaldhunga, Panchthar, Sankhuwasabha, Solukhumbu, Sunsari, Taplejung, Terhathum, Udayapur**
    - **Madhesh: Bara, Dhanusha, Mahottari, Parsa, Rautahat, Saptari, Sarlahi, Siraha**
    - **Bagmati: Bhaktapur, Chitwan, Dhading, Dolakha, Kathmandu, Kavrepalanchok, Lalitpur, Makwanpur, Nuwakot, Ramechhap, Rasuwa, Sindhuli, Sindhupalchok**
    - **Gandaki: Baglung, Gorkha, Kaski, Lamjung, Manang, Myagdi, Nawalpur (Nawalparasi East), Parbat, Syangja, Tanahun**
    - **Lumbini: Arghakhanchi, Banke, Bardiya, Dang, Gulmi, Kapilvastu, Nawalparasi (Bardaghat Susta West), Palpa, Pyuthan, Rolpa, Rupandehi**
    - **Karnali: Dailekh, Dolpa, Humla, Jajarkot, Jumla, Kalikot, Mugu, Rukum (Eastern), Salyan, Surkhet**
    - **Sudurpashchim: Achham, Baitadi, Bajhang, Bajura, Dadeldhura, Darchula, Kailali, Kanchanpur, Doti**

Blood Groups:

- **A+, A-, B+, B-, AB+, AB-, O+, O-**

Units and Wards:

- **Units: Unit 1, Unit 2, Unit 3 per key departments**
- **Wards: General Ward, ICU, NICU, PICU, Maternity, Surgery, Orthopedics**

Rooms/Beds:

- **For each ward, create 10 rooms with 4 beds each (configurable)**

Departments (20+ example including specified):

- **Operation Theater**
- **Laboratory**
- **Radiology**
- **X-Ray**

- **CT Scan**
- **MRI**
- **Nursing**
- **Pharmacy**
- **Emergency**
- **OPD**
- **ICU**
- **Pediatrics**
- **Gynecology**
- **Cardiology**
- **Orthopedics**
- **ENT**
- **Dermatology**
- **Dental**
- **Gastroenterology**
- **Neurology**

Medical Services (20+ including specified):

- **Emergency**
- **General Medicine**
- **Orthopedics**
- **ENT**
- **Cardiology**
- **Neurology**
- **Gastroenterology**
- **ICU**
- **Dermatology**
- **Dental**
- **Pediatrics**

- **Gynecology**
- **Psychiatry**
- **Radiology**
- **Laboratory**
- **Physiotherapy**
- **Nephrology**
- **Oncology**
- **Urology**
- **Pulmonology**

Doctors (50+):

- **Randomly generate 50–100 with names like Dr. Sushil Koirala, assign departments and units, daily quotas, and schedules.**

Medicines (500+):

- **Use a CSV or Faker seeder to create at least 500 medicines. Include common generics and brands, forms (tablet, syrup, injection), strengths, prices, tax.**

Lab Tests (sample 50+):

- **CBC, Hemoglobin, Platelet Count, WBC Differential, ESR, Blood Glucose (F, PP, R), HbA1c, Lipid Profile, LFT, RFT, Serum Electrolytes, Thyroid Profile, CRP, Dengue NS1/IgM/IgG, Widal, HBsAg, Anti-HCV, HIV, Urinalysis, Urine Culture, Blood Culture, PT/INR, aPTT, D-Dimer, Serum Calcium, Vitamin D, Ferritin, etc.**

Insurance Companies (example ~10):

- **Nepal Life Insurance Health, National Insurance Company, Shikhar Insurance Health, NLG Insurance, Siddhartha Insurance, Himalayan General Insurance, Prime Life Insurance (health products), Rastriya Beema Company (health), IME General Insurance, Sagarmatha Insurance (health riders)**

Management command example (seed_medicines.py):

```
from django.core.management.base import BaseCommand
from pharmacy.models import Medicine
from faker import Faker
import random

FORMS = ["Tablet","Capsule","Syrup","Injection","Ointment","Drops"]
STRENGTHS = ["5 mg","10 mg","20 mg","50 mg","100 mg","250 mg","500
```

```
mg","5%","10%"]
GENERIC_POOL = [
("Paracetamol","Acetaminophen"),
("Amoxicillin","Amoxicillin"),
("Azithromycin","Azithromycin"),
("Pantoprazole","Pantoprazole"),
("Metformin","Metformin"),
("Amlodipine","Amlodipine"),
("Losartan","Losartan"),
("Atorvastatin","Atorvastatin"),
("Omeprazole","Omeprazole"),
("Cefixime","Cefixime"),

... extend pool

]

class Command(BaseCommand):
help = "Seed 500+ medicines"
def handle(self, *args, **options):
faker = Faker()
created = 0
# Seed from generic pool
for gname, gen in GENERIC_POOL:
for i in range(30): # 30 brands per generic approx
Medicine.objects.create(
name=f"{gname} {random.choice(STRENGTHS)} {random.choice(FORMS)}
{faker.company()}",
generic_name=gen,
form=random.choice(FORMS),
strength=random.choice(STRENGTHS),
tax_percent=random.choice([0,5,13]),
purchase_price=round(random.uniform(2,100),2),
sale_price=round(random.uniform(5,150),2),
is_prescription_required=True
)
created += 1
if created >= 500:
self.stdout.write(self.style.SUCCESS(f"Seeded {created} medicines"))
return
self.stdout.write(self.style.SUCCESS(f"Seeded {created} medicines"))
```

Management command example (seed_doctors.py):

```
from django.core.management.base import BaseCommand
from django.contrib.auth import get_user_model
from doctors.models import Doctor, DoctorAvailability
from settings_app.models import Department, Unit
from faker import Faker
import random, datetime

User = get_user_model()

class Command(BaseCommand):
help = "Seed 50+ doctors"
def handle(self, *args, **options):
faker = Faker()
departments = list(Department.objects.all())
for i in range(60):
email = f"doctor{i+1}@hamrohospital.com"
user = User.objects.create_user(email=email, password="ChangeMe123!", full_name=f"Dr.
{faker.name()}", is_doctor=True, role="doctor")
dep = random.choice(departments)
unit = dep.units.first()
doc = Doctor.objects.create(user=user, department=dep, unit=unit, title="Dr.",
degrees="MBBS, MD", daily_quota=random.randint(20, 50))
# Availability Mon-Fri mornings
for dow in range(0,5):
DoctorAvailability.objects.create(doctor=doc, day_of_week=dow, shift="morning",
start_time=datetime.time(9,0), end_time=datetime.time(13,0),
max_quota=doc.daily_quota)
```

Management command example (seed_departments_services.py):

- **Create the 20+ departments and 20+ services listed above.**

Management command (seed_provinces_districts.py):

- **Insert all 7 provinces and 77 districts with correct mapping.**

Management command (seed_lab_tests.py):

- **Insert 50+ common lab tests with realistic prices.**

Rooms/Beds:

- **For each ward: create N rooms and M beds per room based on constants.**

Blood groups:

- **Insert A+, A-, B+, B-, AB+, AB-, O+, O-**

31) Jazzmin Admin Configuration (Django Admin)

In settings:

```
JAZZMIN_SETTINGS = {
"site_title": "Hamro Hospital Admin",
"site_header": "Hamro Hospital",
"welcome_sign": "Welcome to Hamro Hospital Admin",
"site_brand": "Hamro Hospital",
"show_ui_builder": False,
"icons": {
"accounts.User": "fas fa-user",
"patients.Patient": "fas fa-id-card",
"doctors.Doctor": "fas fa-user-md",
"appointments.Appointment": "fas fa-calendar-check",
"billing.Invoice": "fas fa-file-invoice-dollar",
"laboratory.LabOrder": "fas fa-vial",
"radiology.RadiologyOrder": "fas fa-x-ray",
"pharmacy.Medicine": "fas fa-pills",
},
"topmenu_links": [
{"name": "Dashboard", "url": "admin:index", "permissions": ["auth.view_user"]},
],
"order_with_respect_to":
["settings_app.Department","settings_app.Service","patients.Patient","doctors.Doctor"],
"custom_css": None,
"custom_js": None,
}
```

- **Limit Admin to super_admin and internal staff; do not expose externally.**

32) Templates and UI Components

- **Base layout: container-fluid for dashboards; container for public pages**
- **Components:**
    - **Navbar with sticky-top**
    - **Breadcrumbs**
    - **Cards for summaries**
    - **Tables with .table-striped, .table-hover**
    - **Modals for department details and scanner UI**

    - **Toasts for feedback**
    - **Tooltips for actions**
- **Animations:**
    - **Fade in for modals and cards**
    - **Avoid long or repetitive animations in clinical areas**

33) PDF Templates

- **Use dedicated templates for PDF (e.g., templates/pdf/invoice.html, pdf/lab_report.html)**
- **Load brand colors via inline styles or minimal CSS file**
- **Ensure absolute URLs for images (logo, QR, barcode)**

34) Scanning (QR/Barcode)

Front-end:

- **Add a modal with <video> element**
- **Use jsQR for QR**
- **Use QuaggaJS for barcode (Code128)**
- **On decode, hit /api/patients/{hospital_id} or /api/patients?barcode= to fetch basic info**
- **Provide manual entry fallback**

35) Performance and Scalability

- **PostgreSQL indexes: (doctor_id, date) on Appointment; (invoice_no); (medicine_id, expiry_date) on StockBatch**
- **Use select_related/prefetch_related on list views**
- **Redis caching for public pages lists and dashboard counts**
- **Celery tasks for heavy operations (PDF generation in batch, reports, backup)**
- **Consider read-replica DB if scale grows**
- **File uploads to S3 to offload web servers**

36) Testing

- **Unit tests for models: constraints, methods (availability check, invoice total)**
- **API tests for auth, patients, doctors, appointments**

- **Business rule tests: leave blocks booking, stock cannot go negative**
- **PDF generation smoke tests**
- **E2E smoke with Selenium or Playwright for key flows**
- **Load test appointment/bookings endpoints (locust)**

37) Internationalization (Optional but recommended)

- **Use gettext for strings**
- **Provide Nepali translations for public site and patient dashboard**
- **Date/time localized to Asia/Kathmandu**

38) Example Code Snippets

Generate Patient ID (core/utils.py):

```
import datetime, random
def generate_hospital_id():
year = datetime.date.today().year
serial = random.randint(100000, 999999) # replace with sequence from DB in production
return f"HH-{year}-{serial}"
```

QR/Barcode signals (patients/signals.py):

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Patient
from core.utils import generate_qr_png, generate_barcode_png

@receiver(post_save, sender=Patient)
def create_codes(sender, instance, created, **kwargs):
if created:
instance.qr_image.save(f"{instance.hospital_id}.png",
generate_qr_png(f"HH:{instance.hospital_id}"))
instance.barcode_image.save(f"{instance.hospital_id}.png",
generate_barcode_png(instance.hospital_id))
```

Invoice totals (billing/models.py):

```
def recalc(self):
amounts = [item.amount for item in self.items.all()]
self.total = sum(amounts)
self.grand_total = self.total + self.tax - self.discount
self.due_amount = max(self.grand_total - self.paid_amount, 0)
self.save(update_fields=["total","grand_total","due_amount"])
```

39) Validation Rules (selected)

- **Patient:**
    - **email unique if provided**
    - **phone pattern ^+?977[9]\d{8}$ or local 98/97 numbers normalized to +977**
    - **age derived if DOB provided**
- **Appointment:**
    - **must fall within DoctorAvailability**
    - **ensure quota not exceeded: count confirmed/check-in for given doctor/date/shift**
- **Pharmacy:**
    - **before selling, ensure stock available and batch not expired**
- **Billing:**
    - **invoice cannot be "paid" unless paid_amount >= grand_total**

40) Error Pages and Logging

- **templates/errors/403.html, 404.html, 500.html**
- **Logging config: rotate logs; per-module loggers**
- **Correlation ID middleware to trace requests; include in error pages for support**

41) Payment Integration Notes

eSewa:

- **Create sandbox and production merchant credentials**
- **Payment init endpoint: creates a payment request with amount and callback URL**
- **Callback verification: validate hash/signature; record txn_id; set invoice paid**
- **Retry mechanics and idempotency key per invoice**

QR Payment:

- **If using interoperable QR: generate QR string with merchant/payment details**
- **Confirm payment via PSP dashboard or webhook if available**
- **Always support manual override with manager approval and audit trail**

42) Backups (scripts)

scripts/backup.sh:

```
#!/usr/bin/env bash
set -e
DATE=$(date +%F-%H%M%S)
pg_dump $DATABASE_URL | gzip > backups/db-$DATE.sql.gz
tar -czf backups/media-$DATE.tar.gz media/
```

Optionally upload to S3

```
aws s3 cp backups/db-$DATE.sql.gz s3://your-bucket/
```

```
aws s3 cp backups/media-$DATE.tar.gz s3://your-bucket/
```

scripts/restore.sh: inverse operations with confirmation prompt.

43) Deployment (Docker)

Dockerfile (web):

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1
WORKDIR /app
RUN apt-get update && apt-get install -y build-essential libpq-dev libpango-1.0-0
libpangoft2-1.0-0 libffi-dev
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
CMD ["gunicorn", "hamro_hospital.wsgi:application", "-b", "0.0.0.0:8000", "-w", "4"]
```

docker-compose.yml (dev):

- **services: web, db (postgres), redis, worker (celery), beat (celery beat), nginx**

44) Role-Specific Dashboards: Permissions and Menus

- **Super Admin: master menu with system settings, backups, all modules**
- **Doctor: patients search/scan, schedule, leave, referrals, prescriptions, orders**
- **Nurse: ward list, vitals entry, tasks**
- **Registration Counter: patient CRUD (critical fields), appointments**
- **Pharmacy: prescriptions queue, dispensing, stock**
- **Laboratory: orders, sample collection, results, verification**

- **Radiology: orders, scheduling, reporting**
- **Blood Bank: donors, inventory, issues**
- **OT: bookings, procedures, notes**
- **Admission & Discharge: admissions, transfers, discharges, bed board**
- **Cash/Accounts: invoices, payments, day close**
- **Insurance: policies, claims**
- **Medical Records: encounters, documents**
- **Patient: profile, card, records, appointments, bills, insurance, admissions, pharmacy**

45) Notifications UX

- **Bell icon with unread count**
- **Notifications dropdown with latest 10**
- **"View all" page with filters**
- **Mark read/unread**
- **Email/SMS toggles in user settings**

46) Data Export and Interop

- **CSV/XLSX export for lists with current filters**
- **Import tools for medicines and tests from CSV**
- **OpenAPI schema auto-generated for APIs**
- **Consider FHIR mapping (future) for interoperability**

47) Accessibility and Mobile

- **All primary actions reachable within 2 taps on mobile**
- **Larger tap areas; table views convert to stacked cards on small screens**
- **Avoid red for normal buttons; use accent red for alerts/warnings only**

48) i18n Content Examples

- **Slogan: "हाम्रो अस्पताल — स्वास्थ्य नै सबैभन्दा ठूलो धन हो।"**
- **Labels in Nepali for public pages; keep dashboards bilingual if desired**

49) Audit and Compliance

- **Audit every update to patient demographic data, results, invoices, admissions**
- **Immutable logs (write-only) for critical operations; expose audit trail in admin**
- **Access reports: who viewed a patient record (optional, if required by policy)**

50) Sample Fixtures (JSON excerpts)

You can create fixtures/choices_provinces_districts.json:

```
[
{"model": "core.province", "pk": 1, "fields": {"name": "Koshi"}},
...
]
```

Or rely on seed_provinces_districts.py as management command for maintainability.

For blood groups, services, departments, lab tests, use similar fixtures or seeders as commands.

51) Dark Mode in Dashboard

- **Use same toggle code as public**
- **Define CSS variables to ensure charts/tables swap palettes**
- **Store user preference in localStorage; optionally sync to user profile**

52) Doctor Leave Message

When DoctorLeave active for today:

- **Show red alert on doctor profile and appointment form:**
  “Doctor is currently on leave.”

53) Barcode and QR Placement

Must appear on:

- **Patient Card**
- **Bills / Receipts**
- **Reports (Lab, Radiology)**
- **OPD Tickets**
- **Prescriptions**
- **Pharmacy Bills**

Standardize component for printing header with logo + QR/Barcode so all PDFs are consistent.

54) OPD Ticket Template

- **Ticket No**
- **Patient Name, Age/Gender, Hospital ID**
- **Department/Doctor**
- **Date/Time**
- **Barcode of ticket_no for quick scan at check-in**

55) Doctor-Only Editable Fields

- **Schedule (DoctorAvailability)**
- **Quota (Doctor.daily_quota)**
- **Availability (e.g., quick toggle "Accepting OPD today")**
- **Leave (DoctorLeave)**
- **Back-end enforces user==doctor.user for updates**

56) Cache Strategy

- **Cache service lists, department lists (15m)**
- **Cache doctors list (5m); bust on doctor update**
- **Dashboard KPIs cached per role (30–60s) with keys per user or per department**

57) Select Code Maps and Enums

- **Genders: M, F, O**
- **Payment methods: cash, esewa, qr**
- **Appointment statuses: requested, confirmed, checked_in, completed, no_show, cancelled**
- **Referral statuses: new, in_progress, completed**
- **Lab statuses: new, collected, processing, verified**
- **Radiology statuses: new, scheduled, completed, reported**
- **Invoice statuses: unpaid, partial, paid, void**

58) Global Search

- **For admin roles: global search bar across patients, invoices, referrals, orders**
- **Implement pointer to direct entity pages**

59) Consent and Policies

- **Patient registration flow to include consent to data processing and hospital policies**
- **Store acceptance timestamp**

60) Go-Live Checklist (abbreviated for execution)

- **Settings configured for production (DEBUG=False, ALLOWED_HOSTS, SECURE_* on)**
- **Migrations applied; super admin created**
- **Roles and permissions assigned**
- **Payment and SMS gateways live keys set; webhooks reachable over HTTPS**
- **Email domain verified**
- **Backups scheduled and tested**
- **Logging/monitoring dashboards live**
- **Staff training completed**
- **Data import (if migrating from old system)**
- **Final UAT sign-off**

61) Example Front-End Snippets

Navbar notification icon (public):

**<li class="nav-item dropdown"> 3 <ul class="dropdown-menu dropdown-menu-end" aria-labelledby="notifDropdown"> New service added Holiday notice <hr class="dropdown-divider"> View all **

62) Error Messages and Alerts Copy

- **OTP sent: “We’ve sent a 6-digit code to your number. It expires in 10 minutes.”**
- **OTP invalid: “That code doesn’t match. Please try again.”**
- **Doctor leave: “Doctor is currently on leave.”**
- **Stock low: “Only X units left in stock. Adjust quantity.”**

63) Admin Data Importers

- **CSV upload in Jazzmin or custom admin for medicines/tests**
- **Columns validated; preview before commit**
- **Rollback on failure; log row-level errors**

64) Encounter and Records

- **Doctor creates Encounter when seeing a patient**
- **Add vitals, diagnoses, and prescriptions**
- **Attach scans or documents to encounters**
- **Records visible to patient except internal-only notes**

65) Throttling and Axes

- **django-axes: lockout after 5 failed logins per IP/email for 10 minutes**
- **Custom per-endpoint throttles for OTP generation and payment attempts**

66) DRF Settings

```
REST_FRAMEWORK = {
"DEFAULT_AUTHENTICATION_CLASSES": [
"rest_framework.authentication.SessionAuthentication",
"rest_framework.authentication.TokenAuthentication",
],
"DEFAULT_PERMISSION_CLASSES": [
"rest_framework.permissions.IsAuthenticated",
],
"DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
"PAGE_SIZE": 25,
"DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",
}
```

Add drf-spectacular for OpenAPI docs (/api/schema, /api/docs).

67) Celery Tasks

- **Send emails/SMS async**
- **Generate PDFs in bulk**
- **Nightly bed charges for admitted patients**
- **Backup scheduler**
- **Report scheduler**

- **Appointment reminders**

68) Content Security Policy (CSP)

- **Limit script-src to self, CDN for Bootstrap/Icons**
- **Allow camera only for scanner pages (permissions policy)**
- **img-src self data: for QR/barcodes**

69) Indexes and Migrations Notes

- **Add index_together on Appointment(doctor,date)**
- **UniqueConstraint for BedAllocation non-overlap can be enforced via validation + db exclusion constraint (PostgreSQL EXCLUDE with GIST on timespans) if advanced; or ensure in app logic with locks.**

70) Printing

- **Print stylesheets for tickets and cards**
- **Hide navigation and controls in print**
- **Ensure QR/Barcode high contrast**

71) Sample Forms (Django)

patients/forms.py:

```
from django import forms
from .models import Patient

class PatientProfileForm(forms.ModelForm):
class Meta:
model = Patient
fields = ["date_of_birth","gender","blood_group","phone","email","address"]
def clean_phone(self):
phone = self.cleaned_data["phone"]
# normalize +977...
return phone
```

doctors/forms.py:

```
class DoctorAvailabilityForm(forms.ModelForm):
class Meta:
model = DoctorAvailability
fields = ["day_of_week","shift","start_time","end_time","max_quota"]
```

72) Seed Execution Order

- **python manage.py migrate**
- **python manage.py seed_provinces_districts**
- **python manage.py seed_blood_groups**
- **python manage.py seed_departments_services**
- **python manage.py seed_rooms_beds**
- **python manage.py seed_lab_tests**
- **python manage.py seed_medicines**
- **python manage.py seed_doctors**

73) Example Department Modal (HTML)

**   <h5 class="modal-title" id="deptTitle">Radiology</h5> <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>   High-quality imaging services... Room: B-102 Phone: 01-xxxxxxx Email: radiology@hamro.com   **

74) Appointment Availability Service (pseudo)

- **Input: doctor_id, date**
- **Load DoctorAvailability for that day_of_week**
- **Exclude if DoctorLeave covers date**
- **Count existing confirmed/checked_in appointments in that shift**
- **Return available time slots (15-min or 30-min steps as configured)**

75) Dashboard Recent Activities

- **Log: created patients, invoices, results verified, admissions/discharges, referrals completed**
- **Show last 20 for role scope**

76) Finance/Accounts Notes

- **Taxes configurable per item**

- **Discounts per invoice or per line (with permission)**
- **Daily cash closing report by cashier, payment method breakdown**
- **Export to accounting package CSV**

77) Insurance Claims Flow

- **Submit claim with invoice and attachments**
- **Track status updates**
- **Report: claims by company, approval/rejection rate**

78) Data Privacy

- **Hide patient phone/email from non-essential roles**
- **Allow "mask" view for partial PII in queues (e.g., P***** 98******23)**
- **Log all PII access events if policy requires**

79) Optional Enhancements

- **Queue management with token screens**
- **Patient portal PWA**
- **Auto-scheduler suggestions based on load**
- **FHIR-compliant API for future integrations**

80) Maintenance and Upgrades

- **LTS upgrades planned annually**
- **Dependency updates monthly with test pass criteria**
- **Security updates ASAP with hotfix process**

81) Dark/Light Theme QA

- **Test graph colors, alerts contrast**
- **Buttons: primary (blue); danger (red) only for destructive/alert actions**
- **Forms readable in both themes**

82) User Onboarding

- **First login: prompt to complete profile (patients) or schedule (doctors)**
- **Quick tips modals skippable, remembered per user**

83) Pagination Defaults

- **25 items per page; show page size selector (25, 50, 100)**
- **Server-side order by updated_at desc**

84) Search Indexing

- **PostgreSQL trigram extension for fuzzy patient name search**
- **Index on lower(email) for user search**

85) Attachments and Virus Scanning

- **Optionally integrate ClamAV for uploads**
- **Restrict max file size per type**

86) Environment Variables (sample)

- **SECRET_KEY=**
- **DEBUG=**
- **DATABASE_URL=postgres://user:pass@host:5432/db**
- **REDIS_URL=redis://redis:6379/0**
- **EMAIL_HOST, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD, EMAIL_PORT, EMAIL_USE_TLS**
- **SMS_API_KEY, SMS_SENDER_ID**
- **ESEWA_MERCHANT_ID, ESEWA_SECRET**
- **ALLOWED_HOSTS**
- **MEDIA_STORAGE=s3/local**
- **AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_STORAGE_BUCKET_NAME (if S3)**

87) Health Checks

- **/healthz returns app, db, cache status**
- **Celery worker heartbeat metrics**

88) CI/CD Checklists

- **Lint: flake8, black, isort**
- **Tests: pytest with coverage > 85%**
- **Security: bandit, pip-audit**
- **Build image, run migrations, collectstatic, deploy**

89) Data Dictionary (examples)

- **patients.Patient.hospital_id: unique, format HH-YYYY-XXXXXX**
- **doctors.Doctor.daily_quota: default 40, doctor-editable**
- **appointments.Appointment.status: enum; transitions limited**
- **billing.Invoice: totals recomputed on save**

90) Final Notes

- **Keep UI clean, airy spacing, readable typography (system fonts or Inter)**
- **Use Bootstrap Icons consistently**
- **Keep red strictly for warnings/danger/accent buttons**
- **Match all site accents to your logo palette for that world-class, cohesive feel**

That's the full, production-grade documentation to build Hamro Hospital. If you want, I can split this into code scaffolds per app and generate starter files next, or produce ready-to-import JSON fixtures for districts and initial services/departments.


स्वास्थय नै सम्पत्ति हो
हाम्रो हास्पिटल
HAMRO HOSPITAL